ZETAERP · ZENITH — INTERNAL ENGINEERING

Production-Bug Pull Request SOP

From reproduce to merge-ready — the first time, every time.

Who this is for. Every teammate who fixes a bug on the live product (`erp.zetaclantech.com`) and opens a pull request for it. Follow this and your PR lands on the first review pass — no back-and-forth, no surprises at deploy.

1 · Reproduce + root cause → **2 · Branch** fix/... off master → **3 · Fix** one change → **4 · Verify** throwaway + DB → **5 · CI** green on branch → **6 · PR** fill + ready → **7 · Merge** maintainer

LIVE SYSTEM · MARUTHI = ORG 100000 · ONE LOGICAL CHANGE PER PR · MERGE ≠ DEPLOY

Scope: raising & landing a PR. **Companion docs:** `CONTRIBUTING.md` , `.github/PULL_REQUEST_TEMPLATE.md` , `docs/SOP-commit-and-deploy.md` (deploy runbook).
Classification: Internal — proprietary. Do not share outside the team. **Owner:** maintainer / repo owner.

0 The golden rule

This is a live commercial system with real customer data.

A change is **guilty until proven no-op-safe**. The flagship tenant is **MARUTHI AGENCY (org `100000`)** and it is billing customers right now. Your PR's job is not just "fix the bug" — it is to **prove** the fix is correct *and* that it changes nothing for live tenants unless that is exactly the intent. One PR = one logical change. If you are unsure about a real value (a GSTIN, an HSN code, a price, a tax rate) — **ask the owner. Never invent it.**

1 Before you write any code

Production bugs are fixed at the **root cause**, never at the symptom. A patch that only hides the symptom will be bounced.

1. **Reproduce it reliably.** Get exact steps, the tenant/role it happens for, and the surface (`/apps` , `/qapps` , or both). If you can't reproduce it, gather more data — don't guess.
2. **Find the root cause.** Read the error/stack fully. Trace the bad value back to where it originates. Check what recently changed (`git log` , recent PRs). Fix at the source.
3. **Confirm it's a real production bug**, not a local/environment artifact. Note the expected vs actual behaviour in plain terms — this becomes your PR's "What & why".
4. **Check it isn't already being fixed.** Scan open PRs/issues. If a teammate is on the same area, coordinate — don't open a duplicate or a conflicting fix.

Edit the local mirror, not the container

Work in the git checkout at `C:\Users\ASUS\zenith-erp\component\<module>\…` . It mirrors all `zenith-erp-*` components. Use your editor's search/edit here — do **not** `docker exec … grep/cat` for routine work, and never deploy from your branch yourself. The maintainer deploys after merge.

2 The non-negotiables

Breaking any of these can leak customer data, corrupt billing, or take the site down. They are checked on every PR.

1. **Never commit secrets** — no passwords, API keys, tokens, private keys, or real `.env` . Committed a secret by accident? Tell the owner *immediately* — it must be rotated, not just deleted.
2. **Never expose** security / user / password / customer data in code, commit messages, issues, PRs, or screenshots.
3. **Never fabricate** business/legal data — GSTIN, HSN, PAN, prices, stock, tax rates. Ask the owner for the real value.
4. **Billing paths must be no-op-safe for live tenants.** Verify before/after on a **throwaway tenant**, never on MARUTHI (`100000`) or any real tenant.
5. **Clean up all test data** — seed with a unique prefix, verify, purge by exact ids (`docs/infra/purge-tenant.sh`). Confirm zero residue.
6. **Never alter or renumber an already-issued bill / invoice.**
7. On the live DB, **DELETE only by exact captured ids**, never by a class predicate. An unexpected FK block is a diagnostic — investigate, never force through.
8. **Never** stage with `git add -A` / `git add .` — stage explicit paths (or `git add -u`). A stray file in the diff gets the PR bounced.
9. **Never push to `master` .** All changes go through a PR. (A CI guard fails any direct or force push to master.)

3 Branch → fix → commit

3.1 — Arm the git hooks (once per clone)

```
# installs the pre-commit lint guards
sh docs/infra/install-git-hooks.sh
```

3.2 — Branch off the latest master

```
git fetch origin
git switch -c fix/<area>-<short-desc> origin/master
# e.g. fix/sales-tax-inclusive-totals, fix/pos-drawer-reconciliation
```

3.3 — Make exactly one logical change

Fix the root cause only. Leave the data layer and write paths untouched *unless that is the change*. Match the style, naming, and comment density of the surrounding code. No "while I'm here" refactors — they make the diff unreviewable and delay the merge.

3.4 — Commit with explicit paths and a real message

```
git add component/zenith-erp-sales/service/zenith/sales/SalesLineServices.xml
# stage every path by name — NEVER `git add -A` / `.`
git commit
```

Message format: a `type(module): subject` summary line, then a body that explains **the root cause** and **how you verified it**. AI-assisted commits end with the repo's `Co-Authored-By:` trailer — match the existing history style.

> **Good commit body, condensed:** *"#23 stored pre-tax totals; the receivables/aging readers expect GST-inclusive values, so AR under-reported. Store the tax-inclusive grand in `invoiceTotal` / `unpaidTotal`; payable = grand + round-off − payments. Verified: real Accounting invoice + aging + GSTR-3B reconcile on a throwaway tenant."*

4 Architecture rules that get PRs bounced

These are the footguns the reviewer looks for first. Respect them and your diff sails through.

Area	Rule
Org isolation CRITICAL	Every list / detail / JSON transition must scope by `organizationPartyId`. Screen transitions run *outside* the active-org entity filter, so a `sendJsonResponse` / JSON-lookup transition must scope the org **explicitly** (even under `.disableAuthz()`). Use a fail-closed sentinel (`__NO_ORGANIZATION__`) when there is no active org. Use the org-scoped `PartyPicker` include — **never** the stock `searchPartyList` (cross-tenant leak).
Two surfaces	`/apps` = Bootstrap 3, `/qapps` = Quasar SPA. They do *not* share utility classes. `/qapps` **strips `<script>` and `<style>` tags** — bootstrap client JS there via an `<img onload>` attribute → external `<script src>`. If your fix touches UI, it must render on **both**.
Moqui gotchas	`<else-if>` must *nest* its actions (a self-closing `<else-if/>` does not branch). `type="date"` is day-granular → add a `.orderBy("-lastUpdatedStamp")` tiebreaker. Inside `<![CDATA[ ]]>` Groovy use literal `&&` and `<`; in plain XML attributes use `&&` / `<`. Write transitions must be POSTed with `moquiSessionToken`.
Invoice totals	System-wide convention: `invoiceTotal` / `unpaidTotal` store the **tax-inclusive grand** (taxable + tax); round-off is separate. All receivables / aging / TCS / GSTR consumers read it that way. Do not store pre-tax totals.
Static assets	CSS/JS under `zenithstatic/` are cache-busted by a `?v=N` query in `ZenithApp.xml` — bump it when you change them, or browsers serve stale files.

Run the lint guards before you push

The pre-commit hook runs them; the **same guards run in CI and cannot be skipped**. A red check blocks the merge — fix the cause, never `--no-verify` around it.

Guard	Catches
`lint-party-picker`	a non-org-aware party lookup (cross-tenant leak)
`lint-css-comments`	a stray `*/` in a CSS comment (kills the next rule)
`lint-cross-module-transitions`	an unregistered cross-module transition call
`lint-form-dom-collisions`	a form control named after a Vue-render DOM property

5 Verify the way the maintainer will

"It looks right" is not verification. Prove it on ground-truth state — caches lie.

- **Prove the bug is gone** and the obvious adjacent cases still work (no regression).
- **Verify on a throwaway tenant, never MARUTHI.** Seed an isolated org with a unique prefix, exercise the path, then purge. For write paths, **assert the DB changed as expected** — don't trust a page echoing your own edit.
- **Billing / auth / isolation paths:** prove the **no-op for MARUTHI** — capture the relevant state before and after and show it's unchanged (or changed only as intended).
- **UI:** confirm it renders on **both** `/apps` and `/qapps`.
- **Clean up** every seeded row and confirm zero residue.

Three verification traps that have bitten the team

- **Assert fixtures exist *before* you act** — the service runner can swallow errors, so a failed reset passes silently and you "verify" against stale data.
- **Assert on the DB, not a rendered page** — a screen can echo your own static copy from cache.
- **A DB write is not proof of rendering** — and a `200` on a SPA shell is not proof the screen works.

6 Run the acceptance CI on your branch

The staging-acceptance workflow boots a clean image, overlays **your** component tree, and runs the org-isolation, billing-golden, immutability, and render-smoke asserts. Run it **before** you ask for review and paste the run link in the PR.

```
# push your branch first, then dispatch against its head SHA
git push -u origin fix/<area>-<short-desc>
SHA=$(git rev-parse HEAD)
gh workflow run staging-acceptance.yml --ref master -f candidate_sha=$SHA

# add a profile when your fix touches those modules:
#   -f contacts_profile=sales     (Sales runtime suite)
#   -f contacts_profile=ui|security  (Contacts profiles)
```

Why `candidate_sha`? It overlays your branch's code but runs *master's* workflow, asserts, and fixtures — so it validates your fix against the real acceptance suite **without merging**. The repo-guards (lint) also run automatically on every push. **Both must be green** before review.

7 Open the PR so it merges on the first pass

Open it as a **draft** while you finish verifying; mark it **ready for review** only once every section is filled and CI is green. A PR left in draft cannot be merged.

```
gh pr create --base master --draft --title "fix(<module>): <subject>"
# fill the template, confirm CI green, then:
gh pr ready
```

Fill every section of the PR template — honestly

Section	What the maintainer expects to see
What & why	The bug and its **root cause**, plus the linked issue. Not "fixes a bug" — the actual mechanism.
Screens / modules touched	Exact component paths and which surfaces (`/apps` , `/qapps` , both).
How it was verified	Ground-truth evidence: throwaway tenant, asserted DB state, then purged; the **MARUTHI no-op proof** for billing paths; the **CI run link**; both surfaces if UI.
Safety checklist	Every box ticked (see below). An unticked or hand-waved box is a blocker.

A bug-report table makes a fix PR reviewable in minutes

For a multi-file fix, add a short table — the format that gets fix PRs merged fast:

File	Issue / root cause	Correction & verified regression
`SalesLineServices.xml`	Payment guard rejected credit notes but allowed debit notes	Reject both in the shared payable guard. Nonzero notes produce no payment links.
`SalesInvoiceServices.xml`	String-sorted sequence misattributes tax past 99 lines	Resolve each item by the id returned at creation. 105-line invoice reconciles to header GST.

The safety checklist (from the PR template)

- ☐ **No secrets** committed (no passwords, API keys, tokens, private keys, real `.env`).
- ☐ **No customer / user / security data** in the diff, commits, or screenshots.
- ☐ **No fabricated** business/legal data (GSTIN, HSN, PAN, prices, stock, tax).
- ☐ **Org isolation** preserved — every list/detail/JSON path scopes by `organizationPartyId`.
- ☐ **Billing-path safe** — verified no-op-safe on a throwaway tenant, not a real one.
- ☐ **No test data left behind** — seeded with a prefix, verified, purged by exact ids.
- ☐ **No issued bill/invoice** altered or renumbered.
- ☐ **Lint guards pass** locally and in CI (repo-guards).
- ☐ Renders correctly on **both** `/apps` and `/qapps` (if UI).
- ☐ **Staged explicit paths** (no `git add -A` / `.`).

Flag a DEPLOY-GATE when it matters

If the fix touches a **billing path**, changes a **stored convention** (e.g. how a total is stored), or alters **auth / isolation**, say so prominently in the PR ("⚠️ Deploy-gate: …"). It tells the maintainer to run the MARUTHI reverify before the change goes to production. Hiding it just moves the surprise to deploy day.

8 Keep it mergeable (rebase discipline)

Master moves. A PR that has drifted goes `CONFLICTING` and can't be merged until it's reconciled.

- **Rebase onto the current `origin/master` HEAD** before you ask for review, and again whenever master advances under you.
- **Resolve conflicts by taking the correct side** — then re-run the acceptance CI on the new head.
- **If a teammate fixed the same area**, converge on their work — never revert someone else's commit to force yours through. If your edits are identical, take the superset.

```
git fetch origin
git rebase origin/master
# resolve, then:
git push --force-with-lease     # your OWN fix/ branch only — never master
```

Reviewer's note from a real case: two people fixed the same invoice-total bug in parallel. The second PR went `DIRTY` only because the lines were changed to the *same* values on both sides. The fix was a one-minute rebase taking the newer branch's QA files — *not* a logic change. Rebasing early avoids this entirely.

9 The merge bar — what the maintainer checks

Pre-empt all of these and there is nothing left to block the merge.

- ☐ Mergeable — rebased, `CLEAN`, no conflicts
- ☐ CI green — acceptance + repo-guards, link in the PR
- ☐ Org isolation verified (scoped, fail-closed)
- ☐ MARUTHI no-op proven for billing/auth/isolation
- ☐ One logical change, root cause not symptom
- ☐ Explicit-path staging; no stray files
- ☐ No secrets; no fabricated business data
- ☐ Renders on both surfaces (if UI)
- ☐ Deploy-gate flagged if it applies
- ☐ Marked ready for review (not draft)

Anything touching a billing, auth, or org-isolation path needs a review before merge — don't self-merge those.

10 After merge: deploy is separate and gated

Merge ≠ deploy

Merging to `master` integrates your code; it does **not** put it in front of MARUTHI. The maintainer deploys to production separately, via the gated runbook (`docs/SOP-commit-and-deploy.md`): pre-flight MARUTHI safety → deploy into the container → commit the image → `docker compose restart` (never `up -d`) → poll for 200 → verify both surfaces → record the undo point. Billing changes get a MARUTHI reverify first. Don't expect your merge to be live immediately, and don't deploy it yourself — **server access and credentials are not in the repo.**

11 Why PRs get bounced (so yours won't)

Rejection reason	The fix
Left in **draft**	Mark it ready once filled + CI green.
CONFLICTING / DIRTY with master	Rebase onto current master HEAD; re-run CI.
Symptom fix, not root cause	Trace the bad value to its origin; fix there.
Isolation **not scoped** on a JSON/transition path	Scope `organizationPartyId` explicitly, fail-closed.
Wrong **stored convention** (e.g. pre-tax total)	Follow the tax-inclusive grand convention; check the consumers.
`git add -A` dragged in a **stray file**	Stage explicit paths; review the diff before committing.
No CI evidence / no MARUTHI no-op proof	Run the acceptance workflow; paste the link and the before/after.
More than **one logical change**	Split it into separate PRs.
Fabricated or placeholder **business data**	Ask the owner for the real value; never invent.

12 One-page cheat sheet

Commands

```
sh docs/infra/install-git-hooks.sh   # once

git fetch origin
git switch -c fix/<area>-<desc> origin/master

# ... fix the root cause, one change ...

git add path/to/file.xml             # explicit!
git commit
git push -u origin fix/<area>-<desc>

SHA=$(git rev-parse HEAD)
gh workflow run staging-acceptance.yml \
  --ref master -f candidate_sha=$SHA
# + -f contacts_profile=sales|ui|security

gh pr create --base master --draft
# fill template, CI green, then:
gh pr ready

# master moved? rebase:
git fetch origin && git rebase origin/master
git push --force-with-lease          # your branch
```

Before you click "ready"

- ☐ Reproduced + fixed at root cause
- ☐ One logical change only
- ☐ Explicit-path staging
- ☐ No secrets / no customer data
- ☐ No fabricated GSTIN/HSN/price/tax
- ☐ Org-scoped (lists + JSON paths)
- ☐ Verified on a throwaway tenant, DB-asserted
- ☐ MARUTHI no-op proven (billing/auth)
- ☐ Test data purged, zero residue
- ☐ Both /apps & /qapps render (if UI)
- ☐ Lint + acceptance CI green (link in PR)
- ☐ Template filled; deploy-gate flagged if needed
- ☐ Rebased onto master HEAD; not draft

When in doubt, ask the owner before you push. It is always cheaper than a rollback.